Departamento de Engenharia Informática

Aprendizagem Computacional — LIACD — 2025/2026

Classificação do Nível de Stress

Projeto Final

Modelos: SVM · Rede Neuronal (MLP) · XGBoost

Duarte Prazeres
duartecastromarques@gmail.com
Nº 2023214962

Marília Brito
mariliamontemorb@gmail.com
Nº 2024225969

Maio de 2026

Índice

1 Introdução **2**

2 Background **2**
2.1 Trabalhos na Plataforma Kaggle 2
2.2 Literatura Académica Relacionada 3

3 Dataset e Análise Exploratória **3**
3.1 Descrição do Dataset 3
3.2 Limpeza e Qualidade dos Dados 4
3.3 Transformação do Target 4
3.4 Visualizações EDA 5

4 Metodologia **9**
4.1 Feature Engineering 9
4.2 Pré-processamento 11
4.2.1 Divisão dos Dados 11
4.2.2 Normalização e Prevenção de Data Leakage 11
4.2.3 Lidar com o Desequilíbrio de Classes 11
4.3 Métricas de Avaliação 13
4.4 Modelos Implementados 14
4.4.1 Support Vector Machine (SVM) 14
4.4.2 Rede Neuronal (MLPClassifier) 14
4.4.3 XGBoost: 3.º Modelo 15
4.4.4 Ensemble de Redes Neuronais (PyTorch) 16
4.5 Validação Cruzada 16

5 Resultados e Discussão **17**
5.1 Resultados no Conjunto de Teste (Hold-out) 17
5.2 Matrizes de Confusão 18
5.2.1 Cenário A: Classificação Binária 18
5.2.2 Cenário B: 5 Classes 19
5.2.3 Cenário C: 10 Classes 19
5.3 Curvas de Loss da Rede Neuronal 20
5.4 Análise Crítica por Cenário 20
5.4.1 Cenário A: Classificação Binária 20
5.4.2 Cenário B: 5 Classes 21
5.4.3 Cenário C: Trade-offs de Granularidade (10 Classes) 21
5.5 Comparação entre Modelos 22
5.6 Efeito do Tuning de Hiperparâmetros 22
5.7 Ensemble de Redes Neuronais (PyTorch) 22
5.8 Interpretabilidade Local (SHAP) 23
5.9 Testes de Significância Estatística 24
5.10 Comparação com Trabalhos Existentes 25
5.11 Limitações e Desafios 25

6 Conclusões **26**

1 Introdução

Neste trabalho, propusemo-nos a construir e avaliar um sistema completo de classificação do nível de stress com base em dados comportamentais recolhidos através de smartphones: utilização do ecrã, padrões de sono e atividade física. Para isso, utilizámos o dataset *Sleep, Screen Time and Stress Analysis*, disponível no Kaggle com 15 000 registos sintéticos [7].

Desde o início percebemos que este problema podia ser enquadrado de formas muito diferentes consoante a granularidade de classes que se pretendesse. Em vez de nos limitarmos a um único formato, decidimos atacá-lo sob três perspetivas de complexidade crescente:

- **Cenário A**: classificação binária: *sem stress* (0) vs *com stress* (1);
- **Cenário B**: 5 classes (ver §3.3 para o processo de discretização);
- **Cenário C**: 10 classes (ver §3.3 para o processo de discretização).

Esta estrutura em três cenários permitiu-nos não só avaliar o desempenho dos modelos, mas também estudar como a complexidade do problema de classificação afeta os resultados.

Para cobrir diferentes paradigmas de aprendizagem automática, escolhemos três modelos: **SVM**, **Rede Neuronal (MLP)** e **XGBoost**. Avaliámo-los com **validação cruzada estratificada** (uma técnica que divide os dados em vários subconjuntos e treina/avalia o modelo em rotação, dando estimativas de desempenho mais fiáveis do que uma divisão única), com **otimização de hiperparâmetros** (ajuste sistemático dos parâmetros de configuração de cada modelo para encontrar a melhor combinação possível) e com **testes de significância estatística** (verificação formal de que as diferenças observadas entre modelos não são produto do acaso). O desempenho é reportado em **F1 macro**, uma métrica que mede a qualidade das previsões tratando todas as classes de forma igualitária, independentemente da sua frequência. Os principais resultados que obtivemos foram F1 macro de $\approx$0.95 (Cenário A), $\approx$0.82 (Cenário B) e $\approx$0.58 (Cenário C).

2 Background

Antes de começarmos, consultámos brevemente o que tinha sido feito sobre este tema, tanto na plataforma Kaggle como na literatura académica, para ter uma referência do ponto de partida.

2.1 Trabalhos na Plataforma Kaggle

Na secção *Code* do dataset no Kaggle encontrámos dois notebooks relevantes. Badrnezhad [8] desenvolveu um pipeline de preparação de dados (*ETL*) sobre as mesmas variáveis que o nosso dataset contém, sem avançar para modelação preditiva. Tolstoyj [9] aplicou *Random Forest* em modo de **regressão** sobre dados semelhantes (sono, ecrã, stress), concluindo que as relações entre as variáveis comportamentais e o nível de stress eram fracas.

2.2 Literatura Académica Relacionada

Para investigar a literatura científica relevante sobre a classificação de stress e perturbações do sono, recorreu-se a um agente autónomo de inteligência artificial. Esta metodologia permitiu realizar uma pesquisa abrangente e profunda em diversas bases de dados e fontes digitais, superando barreiras linguísticas e identificando estudos que, de outra forma, poderiam não ser mapeados. Adicionalmente, a automatização desta etapa otimizou significativamente o tempo de computação e pesquisa em comparação com uma revisão manual, viabilizando uma alocação mais eficiente de recursos e tempo para as fases subsequentes e nucleares do projeto.

Como resultado, a nossa pesquisa encontrou três trabalhos particularmente interessantes:

- Huang et al. [10] compararam seis algoritmos de ML (incluindo SVM, XGBoost e ANN) em dados de sono e saúde, reportando **XGBoost** como o melhor com 90.66% de accuracy;
- Lei et al. [11] usaram *Random Forest* para classificação binária de stress a partir de dados de sono, obtendo F1 = 0.98;
- Acosta et al. [12] utilizaram sensores físicos acoplados ao corpo (frequência cardíaca, temperatura, saturação de oxigénio) para prever stress durante o sono, com todos os classificadores a superar 98% de accuracy.

A Tabela 1 resume estes trabalhos para referência.

Tabela 1: Resumo dos trabalhos consultados (resultados em classificação binária).

Trabalho	**Dados**	**Melhor Modelo**	**Acc.**	**F1**
Lei et al. [11]	Kaggle (sono sintético)	Random Forest	–	0.98
Huang et al. [10]	Kaggle (saúde/sono)	XGBoost	90.66%	–
Acosta et al. [12]	Biosensores físicos	Ensemble	>98%	–

3 Dataset e Análise Exploratória

3.1 Descrição do Dataset

Quando abrimos o dataset pela primeira vez, verificámos que continha 15 000 instâncias com 13 variáveis descrevendo o comportamento diário de diferentes utilizadores: tempo de ecrã diário, duração e qualidade do sono, atividade física, consumo de cafeína, *score* de fadiga mental e dados demográficos (idade, género, ocupação). A variável alvo, `stress_level`, é contínua, o que significou logo à partida que teríamos de a transformar em categorias antes de poder treinar qualquer modelo de classificação.

3.2 Limpeza e Qualidade dos Dados

Antes de avançarmos para a análise, verificámos a qualidade dos dados: não encontrámos valores nulos nem registos duplicados, e removemos a coluna `user_id` por ser um identificador sem valor preditivo. O dataset estava limpo à partida, o que é típico de dados sintéticos.

3.3 Transformação do Target

O enunciado define três cenários de classificação com granularidades diferentes. A primeira tarefa foi converter a variável contínua `stress_level` em classes discretas. Para cada cenário, definimos um objetivo e escolhemos uma estratégia de discretização:

- **Cenário A (2 classes)**: separar os dados em "Sem Stress" (0) e "Com Stress" (1). Usámos *equal-frequency binning* com ponto de corte na mediana (7.38), porque o equilíbrio resultante (≈50/50) impede que os modelos fiquem tendenciosos para a classe maioritária.
- **Cenário B (5 classes)**: dividir o stress em cinco níveis de intensidade crescente (0 a 4). Aplicámos também *equal-frequency binning* (`pd.qcut` com $q = 5$), com a lógica de garantir aproximadamente o mesmo número de instâncias por classe.
- **Cenário C (10 classes)**: mesma lógica, mas com dez níveis (`pd.qcut` com $q = 10$), para uma granularidade mais fina.

Com estas estratégias aplicadas, obtivemos a seguinte distribuição de classes e amostras por target:

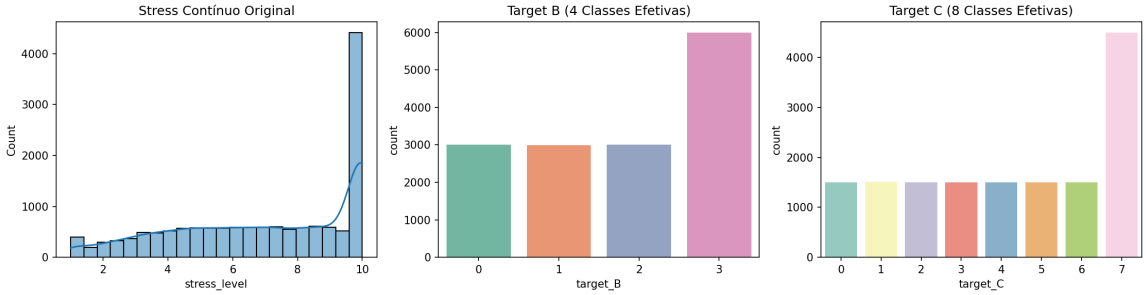


Figura 1: Distribuição dos targets com *equal-frequency binning* para os três cenários. O Cenário A produziu 2 classes equilibradas; os Cenários B e C produziram apenas 4 e 8 classes, respetivamente, em vez das 5 e 10 previstas.

Para o Cenário A o algoritmo funcionou como esperado. Para os Cenários B e C, deparámo-nos com um problema que não antecipámos. Ao visualizar a distribuição resultante, verificámos imediatamente que algo estava errado: o Cenário B tinha apenas **4 barras**, não 5, e o Cenário C tinha apenas **8 barras**, não 10. Ao investigar a causa, percebemos que o problema estava na distribuição do próprio dataset: mais de 20% dos registos têm `stress_level` = 10.0 (o valor máximo). Isto faz com que o 80.º percentil seja 10.0, que é também o máximo absoluto. Para o Cenário B, as fronteiras do 4.º e do 5.º bin são ambas 10.0, ou seja, idênticas. O algoritmo `pd.qcut`, ao detetar esta fronteira duplicada, funde automaticamente os dois bins superiores num só, reduzindo o número

de classes de 5 para 4. No Cenário C o mesmo mecanismo elimina três bins superiores (os percentis 80, 90 e 100 coincidem todos em 10.0), reduzindo de 10 para 8 classes. Como o enunciado exige explicitamente 5 e 10 classes, esta estratégia revelou-se inadequada.

Como ultrapassámos este problema. A raiz do problema estava na natureza do *equal-frequency binning*: ao definir fronteiras com base nos percentis dos dados, o algoritmo não consegue criar bins distintos quando muitos valores se acumulam num único ponto. A solução passou por mudar de estratégia: em vez de garantir **contagens iguais** por classe, passámos a garantir **intervalos de valores iguais** entre classes – *equal-width binning* (`pd.cut`). Com esta abordagem, o intervalo $[\min, \max]$ do `stress_level` é dividido em partes geometricamente iguais, independentemente de quantas instâncias caem em cada parte, produzindo sempre exatamente o número de classes pedido.

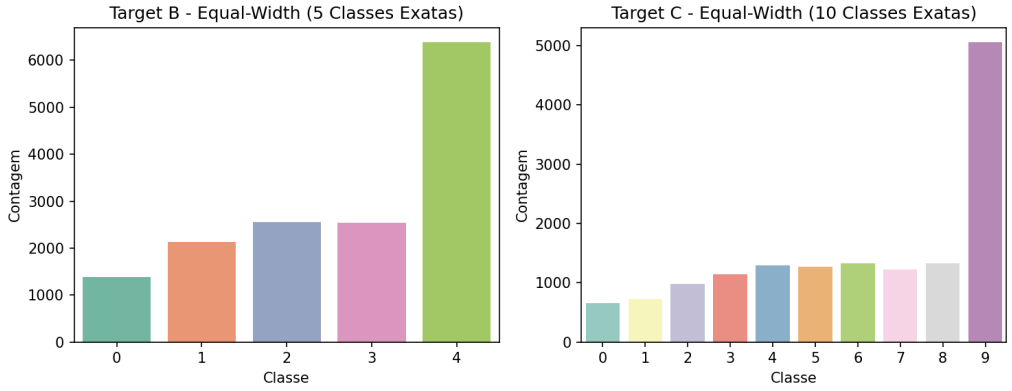


Figura 2: Distribuição dos targets com *equal-width binning*. O Cenário B apresenta exatamente 5 classes e o Cenário C exatamente 10 classes, cumprindo o requisito do enunciado.

Como se pode observar na figura, a nova estratégia cria o número correto de classes. No entanto, introduz um problema diferente: o número de amostras por classe ficou bastante desequilibrado, com a última classe a concentrar muito mais instâncias do que as restantes, consequência direta do efeito de teto do dataset. Este desequilíbrio será necessário endereçar antes do treino dos modelos, testando técnicas de *oversampling* (gerar instâncias sintéticas nas classes minoritárias) ou *undersampling* (reduzir instâncias na classe maioritária), como descrito na secção de pré-processamento (§4.2.3).

3.4 Visualizações EDA

Para compreender os dados antes de os modelar, construímos um conjunto de visualizações exploratórias com um objetivo concreto: responder a três perguntas fundamentais – como se distribuem as variáveis individualmente? quais estão mais correlacionadas com o stress? e as classes que criámos estão razoavelmente equilibradas?

Começámos pelas distribuições univariadas de cada variável numérica (Figura abaixo), que nos deram uma primeira ideia da forma geral dos dados e de eventuais assimetrias. A linha curva sobreposta a cada histograma é a **curva KDE** (*Kernel Density Estimate*): uma estimativa suavizada da forma da distribuição, que ajuda a perceber o padrão geral independentemente do tamanho dos intervalos do histograma.

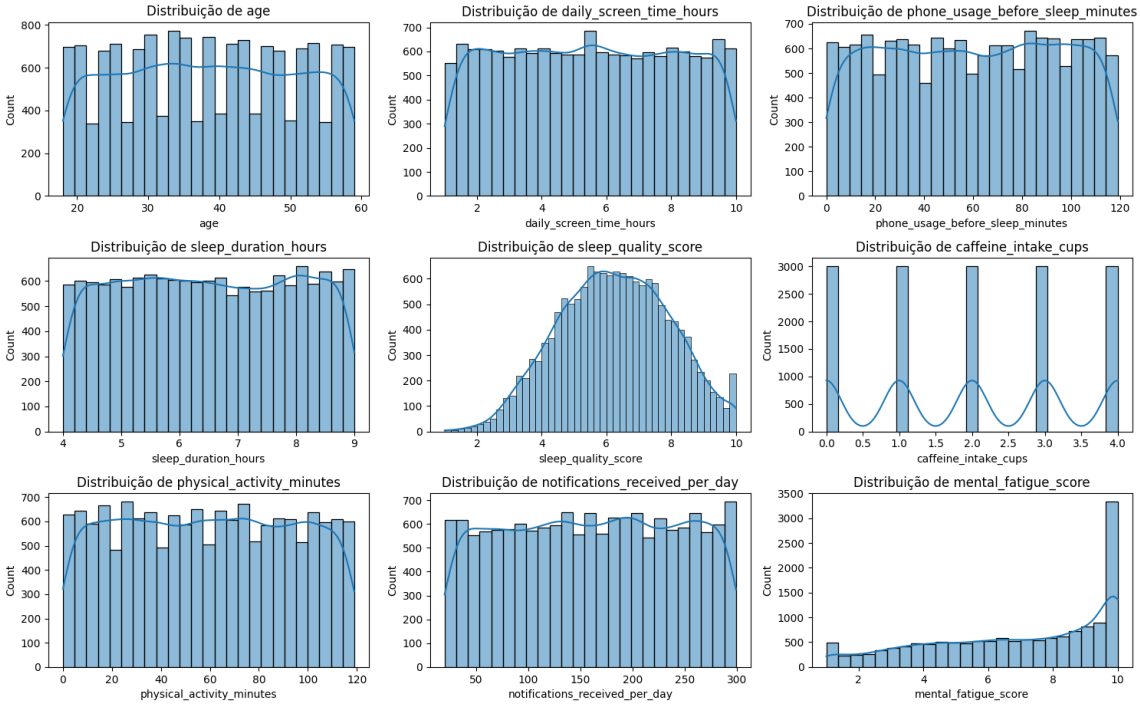


Figura 3: Distribuições univariadas das variáveis numéricas (barras = histograma; linha = KDE suavizado).

Ao analisar os histogramas, observámos dois fenómenos que consideramos relevantes.

O primeiro é um **efeito de teto** (*ceiling effect*) em duas variáveis: `mental_fatigue_score` e `stress_level`. Em ambas, a última barra (valor 10) é claramente a maior. No caso desta variável, o intervalo (9, 10] concentra 4742 registos, mais do dobro do segundo maior intervalo. Esta concentração no valor máximo não é um outlier isolado mas sim um artefacto típico de dados sintéticos, onde a função geradora pode acumular observações nos limites do intervalo definido.

O segundo fenómeno diz respeito ao `caffeine_intake_cups`. Ao observar o histograma, pode parecer que existem valores fracionados como 0.5 ou 1.5 chávenas – o que não faz sentido na realidade. Verificámos os dados e confirmámos que todos os valores são inteiros (0, 1, 2, 3 e 4 chávenas). O que aparece no histograma original são **artefactos visuais**: quando o `histplot` divide o eixo em intervalos automáticos, pode criar barras entre inteiros, e a curva KDE suaviza a distribuição introduzindo valores intermediários. Para ter uma leitura mais correta desta variável, criámos uma segunda visualização com bins discretos:

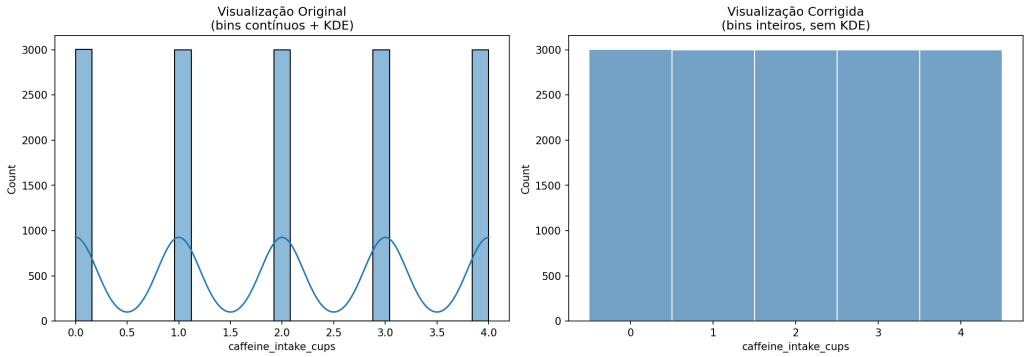


Figura 4: Distribuição de `caffeine_intake_cups`: visualização original com bins contínuos (esquerda) vs. visualização corrigida com bins inteiros (direita). Os dados são apenas inteiros (0–4); as barras a 0.5 e 1.5 na figura da esquerda são artefactos do histograma automático e da curva KDE.

De seguida, construímos dois heatmaps de correlação complementares: um de **Pearson** e outro de **Spearman**. A razão para usarmos as duas métricas foi metodológica: a correlação de Pearson mede apenas relações *lineares*, ao passo que a correlação de Spearman, por operar sobre os *ranks* dos valores em vez dos valores brutos, captura qualquer relação **monotónica**, seja ela logarítmica, exponencial ou de potência. Comparar as duas permite-nos perceber quais as relações têm uma componente não-linear significativa.

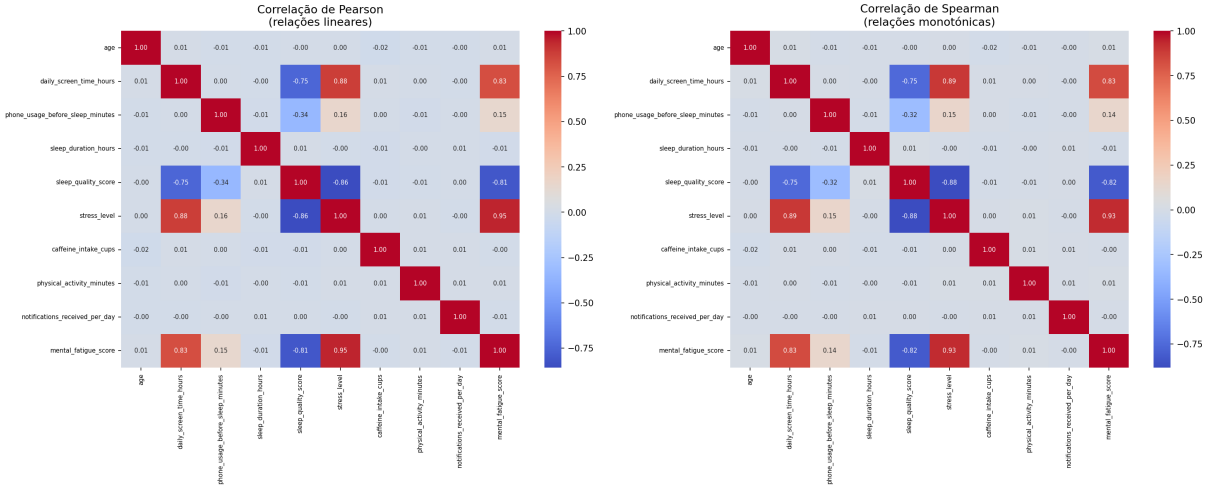


Figura 5: Heatmaps de correlação de Pearson (esquerda) e Spearman (direita).

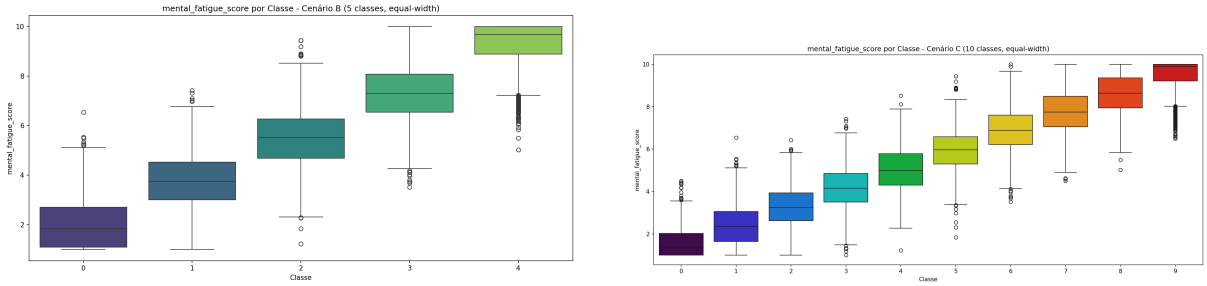
A comparação dos dois heatmaps revelou dois resultados que consideramos importantes. Em primeiro lugar, os três preditores dominantes são os mesmos em ambas as métricas, com valores muito próximos entre si:

- `mental_fatigue_score`: Pearson $r = 0.95$, Spearman $\rho = 0.93$;
- `daily_screen_time_hours`: Pearson $r = 0.88$, Spearman $\rho = 0.89$;
- `sleep_quality_score`: Pearson $r = 0.86$, Spearman $\rho = 0.88$.

Em segundo lugar, e mais interessante, `daily_screen_time_hours` e `sleep_quality_score` apresentam correlação de Spearman *ligeiramente superior* à de Pearson. O que isto significa na prática é que a relação entre estas variáveis e o stress não é perfeitamente proporcional (o que seria uma relação linear), mas cresce de forma monotónica: por exemplo,

mais tempo de ecrã *tende sempre* a corresponder a mais stress, mas o efeito de cada hora adicional pode não ser sempre o mesmo (pode ser maior no início e menor depois de certo ponto, como uma curva logarítmica). Esta descoberta sugere que modelos capazes de aprender fronteiras não-lineares (como o kernel RBF do SVM ou as árvores do XGBoost) têm uma vantagem natural neste dataset. Para o `mental_fatigue_score`, a diferença entre Pearson e Spearman é quase nula, o que indica uma relação mais diretamente proporcional com o stress. Surpreendeu-nos também verificar que variáveis como a atividade física, a duração do sono e o consumo de cafeína apresentam correlações próximas de zero em ambas as métricas, confirmando que a capacidade preditiva do dataset está essencialmente concentrada nas três variáveis-chave acima, algo que influenciou diretamente a nossa estratégia de feature engineering.

Complementámos com um box plot do `mental_fatigue_score` (o preditor mais correlacionado com o stress) por classe, para confirmar visualmente se as classes têm distribuições distintas, tanto no Cenário B (5 classes) como no Cenário C (10 classes).



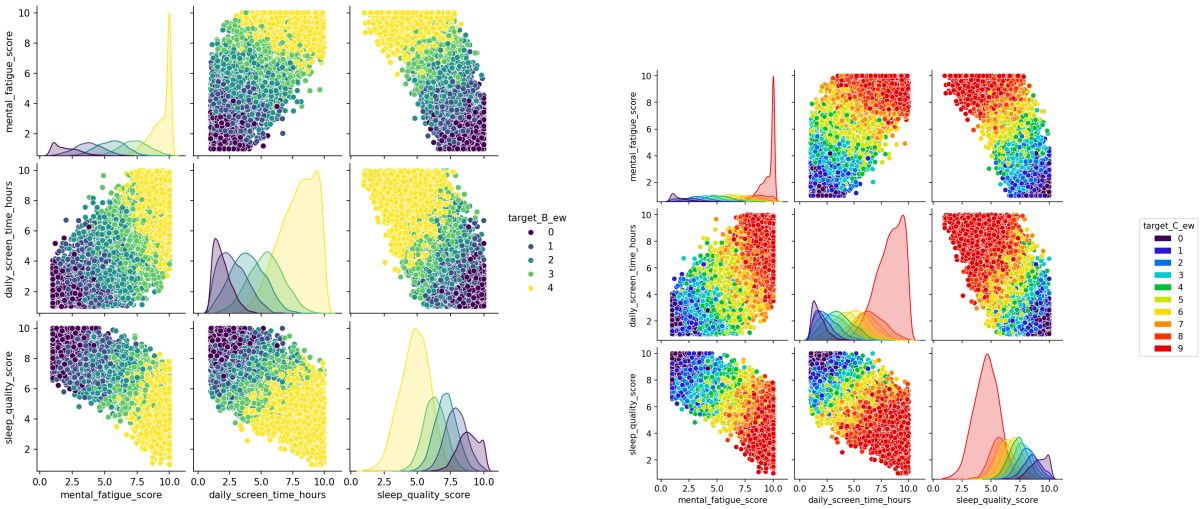
(a) Cenário B (5 classes)

(b) Cenário C (10 classes)

Figura 6: Box plot do `mental_fatigue_score` por classe (equal-width).

Os box plots confirmam aquilo que a análise de correlação já antecipava: há uma separação clara e progressiva das medianas do `mental_fatigue_score` ao longo das classes de ambos os cenários: de baixo stress (classe 0, fadiga baixa) para stress máximo (última classe, fadiga alta). As classes extremas têm distribuições bem separadas, enquanto as classes intermédias apresentam maior sobreposição entre si. No Cenário C, com 10 classes, esta sobreposição é naturalmente mais pronunciada dada a maior granularidade. Este padrão antecipou exatamente o que depois observámos nas matrizes de confusão: os modelos distinguem bem os extremos mas confundem-se nas classes intermédias.

Por fim, gerámos um pairplot das três features mais correlacionadas com o stress (`mental_fatigue_score`, `daily_screen_time_hours` e `sleep_quality_score`) para visualizar as relações entre pares de variáveis e avaliar a separabilidade visual das classes.



(a) Cenário B (5 classes)

(b) Cenário C (10 classes)

Figura 7: Pairplot das três features mais correlacionadas com o stress (equal-width).

O pairplot confirma a estrutura do problema de forma visual. Nos scatter plots fora da diagonal, é percetível uma separação razoável entre as classes, especialmente nas projeções que envolvem o `mental_fatigue_score`: os pontos de classes extremas tendem a ocupar regiões distintas do espaço de features. No Cenário C, a maior granularidade torna a sobreposição entre classes adjacentes mais evidente, antecipando o desafio acrescido de classificação com 10 categorias. Nas distribuições diagonais (KDE por classe), as classes intermédias sobrepõem-se significativamente entre si, mas distinguem-se das extremas. A separabilidade visual, ainda que imperfeita, confirmou-nos que os modelos teriam informação suficiente para aprender, e ajudou-nos a calibrar as expectativas de desempenho antes de avançar para a modelação.

4 Metodologia

4.1 Feature Engineering

Olhando para as features originais do dataset, sentimos que podíamos enriquecer o espaço de atributos com variáveis que capturassem relações que as features brutas não expressavam diretamente. Criámos cinco novas variáveis, cada uma com uma justificação específica:

Feature	Descrição e Justificação
`racio_ecra_sono`	Rácio ecrã/sono. Criámos esta feature porque acreditamos que o efeito do ecrã no stress não é absoluto, mas relativo à quantidade de sono – quem dorme pouco e usa muito o ecrã está em situação muito diferente de quem dorme bastante.
`racio_atividade_ecra`	Atividade física dividida pelo tempo de ecrã. Representa um indicador de estilo de vida equilibrado: valores altos sugerem que a pessoa equilibra o sedentarismo digital com movimento.

`interacao_ecra_fadiga`	Produto de `daily_screen_time_hours` × `mental_fatigue_score`. Criámos esta interação para capturar o efeito multiplicativo entre os dois preditores mais fortes. Acabou por ser **removida** porque apresentava correlação > 0.95 com `mental_fatigue_score`, o que introduziria multicolinearidade sem acrescentar informação nova.
`dorme_menos_6h`	Flag binária: sono < 6h. Adicionámos este indicador clínico porque privação severa de sono é um fator de risco bem documentado para o stress, e uma variável binária captura este limiar de forma mais expressiva do que o valor contínuo.
`sedentario`	Flag binária: atividade < 30min. Pelo mesmo raciocínio, criámos um indicador explícito de sedentarismo, que pode ser um fator de amplificação do stress não facilmente capturado pela variável contínua de atividade.

Para validar as nossas escolhas antes de avançar para a modelação, corremos uma avaliação de importância preliminar com um **Random Forest** simples (50 árvores, Cenário A). Esta é uma técnica agnóstica ao modelo de classificação final, adequada para uma validação exploratória nesta fase.

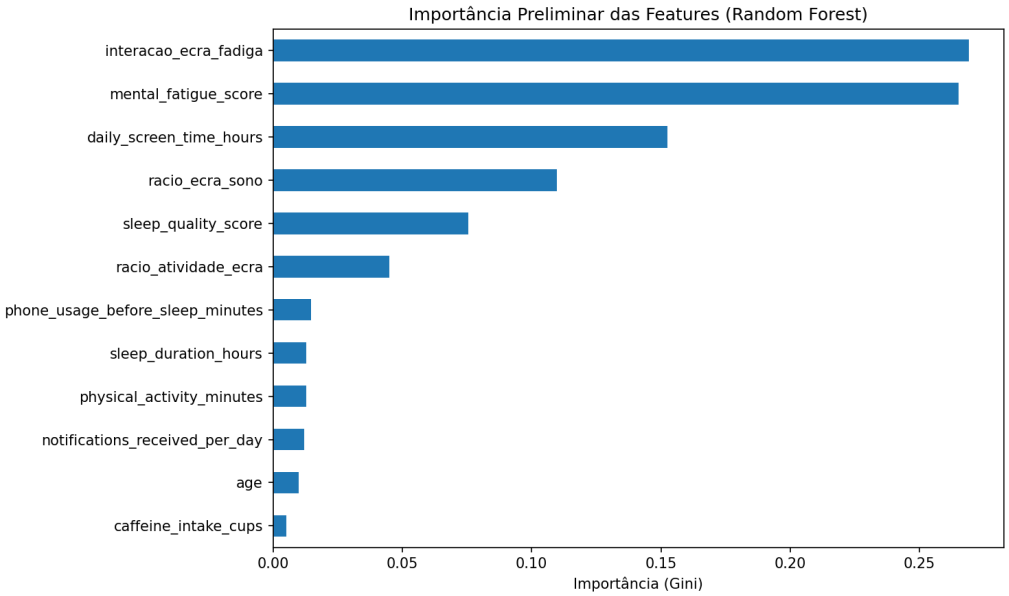


Figura 8: Importância preliminar das features calculada com Random Forest (Cenário A, Gini impurity), antes do filtro de correlação cruzada. A `interacao_ecra_fadiga` aparece no topo por ser um composto das duas variáveis mais fortes, sendo depois removida por multicolinearidade.

O gráfico confirma e quantifica o que a análise de correlação já sugeria. A `interacao_ecra_fadiga` aparece no topo da importância (o que faz sentido, pois

é um produto dos dois preditores mais fortes, concentrando o seu efeito combinado), mas será removida a seguir pelo filtro de correlação cruzada (> 0.95 com `mental_fatigue_score`). Excluindo a interação, o `mental_fatigue_score` domina, seguido do `daily_screen_time_hours` e do `sleep_quality_score`. Os rácios criados (`racio_ecra_sono` e `racio_atividade_ecra`) aparecem com importância não nula, validando a decisão de os criar. As features com importância próxima de zero, como `caffeine_intake_cups` ou `age`, confirmam o que as correlações de Pearson e Spearman já tinham revelado: neste dataset sintético, a capacidade preditiva concentra-se em poucas variáveis-chave.

4.2 Pré-processamento

4.2.1 Divisão dos Dados

Para avaliar os modelos de forma rigorosa, dividimos os dados em três conjuntos distintos, seguindo uma proporção **70/15/15** (treino/validação/teste):

- Treino: 10 500 instâncias, usadas para ajustar os parâmetros dos modelos;
- Validação: 2 250 instâncias, usadas durante a otimização de hiperparâmetros;
- Teste: 2 250 instâncias, reservadas e intocadas até à avaliação final.

Em todas as divisões usámos o parâmetro `stratify`, que garante que a proporção de classes é mantida em cada conjunto, decisão fundamental para que a validação cruzada seja justa e representativa.

4.2.2 Normalização e Prevenção de Data Leakage

Decidimos normalizar todas as features com `StandardScaler`, porque modelos como o SVM são muito sensíveis à escala das variáveis e a sua performance diminui significativamente sem normalização. O passo crítico foi garantir que o scaler era *exclusivamente* calculado sobre os dados de treino (`fit_transform`) e apenas aplicado, sem recalcular, nos conjuntos de validação e teste (`transform`). Fizemo-lo assim para evitar *data leakage*: se calculássemos a média e o desvio padrão com base em todos os dados incluindo validação e teste, estaríamos a dar ao modelo informação sobre dados que ainda não deveria ter visto, inflacionando artificialmente a performance. Para reforçar esta proteção durante a validação cruzada, encapsulámos o scaler num `Pipeline` do scikit-learn, que repete automaticamente este processo correto em cada fold.

4.2.3 Lidar com o Desequilíbrio de Classes

Nos Cenários B e C, como descrito em §3.3, a estratégia de *equal-width binning* garante o número de classes pedido mas cria um desequilíbrio acentuado: a última classe concentra muito mais instâncias do que as restantes devido ao efeito de teto do dataset. Para compensar, adoptámos **SMOTE** (*Synthetic Minority Oversampling Technique*), que gera instâncias sintéticas para as classes minoritárias por interpolação entre vizinhos próximos no espaço de features, sem descartar dados reais.

Seguindo a recomendação de não saltar diretamente para um balanceamento perfeito, testámos seis níveis de forma gradual — apenas nas classes minoritárias, sem alterar a classe maioritária:

1. **Baseline**: sem resampling (distribuição original do treino);
2. **SMOTE +5%**: cada classe minoritária aumentada em 5%;
3. **SMOTE +10%**: cada classe minoritária aumentada em 10%;
4. **SMOTE +20%**: cada classe minoritária aumentada em 20%;
5. **SMOTE +30%**: cada classe minoritária aumentada em 30%;
6. **SMOTE +50%**: cada classe minoritária aumentada em 50%.

Em todos os casos, o SMOTE é aplicado *apenas ao conjunto de treino*, após a divisão treino/validação/teste, para evitar *data leakage*. Os conjuntos de validação e teste mantêm a distribuição real dos dados.

Para escolher o nível de SMOTE sem incorrer em custo computacional excessivo, usámos uma **Random Forest com hiperparâmetros fixos** como modelo proxy (200 árvores, `class_weight='balanced'`, sem tuning). Esta abordagem é justificada porque a seleção do nível de SMOTE é uma decisão de pré-processamento, independente do modelo final, e a Random Forest fornece uma estimativa robusta da separabilidade das classes. O critério de seleção foi o **F1 macro no conjunto de teste**, que mantém a distribuição real dos dados. Escolhemos um nível por cenário, comum aos três modelos, de forma a garantir que a comparação entre SVM, NN e XGBoost é feita sobre o mesmo conjunto de treino. Os resultados obtidos constam das Tabelas 3 e 4.

Tabela 3: Impacto do SMOTE no Cenário B (5 classes) — modelo proxy: Random Forest.

Nível	n **treino**	**F1 Val**	**F1 Teste**
Baseline	10 500	0.7352	0.7455
SMOTE +5%	10 800	0.7421	0.7445
SMOTE +10%	11 102	0.7372	0.7523
SMOTE +20%	11 705	0.7351	0.7469
SMOTE +30%	**12 307**	**0.7415**	**0.7533**
SMOTE +50%	13 515	0.7380	0.7482

Tabela 4: Impacto do SMOTE no Cenário C (10 classes) — modelo proxy: Random Forest.

Nível	n **treino**	**F1 Val**	**F1 Teste**
Baseline	**10 500**	**0.4658**	**0.4848**
SMOTE +5%	10 845	0.4616	0.4682
SMOTE +10%	11 193	0.4640	0.4725
SMOTE +20%	11 888	0.4705	0.4812
SMOTE +30%	12 585	0.4697	0.4776
SMOTE +50%	13 979	0.4781	0.4843

Para ilustrar visualmente o efeito do SMOTE +30% no Cenário B, a Figura 9 mostra a distribuição de amostras por classe no conjunto de treino antes e depois da aplicação do oversampling.

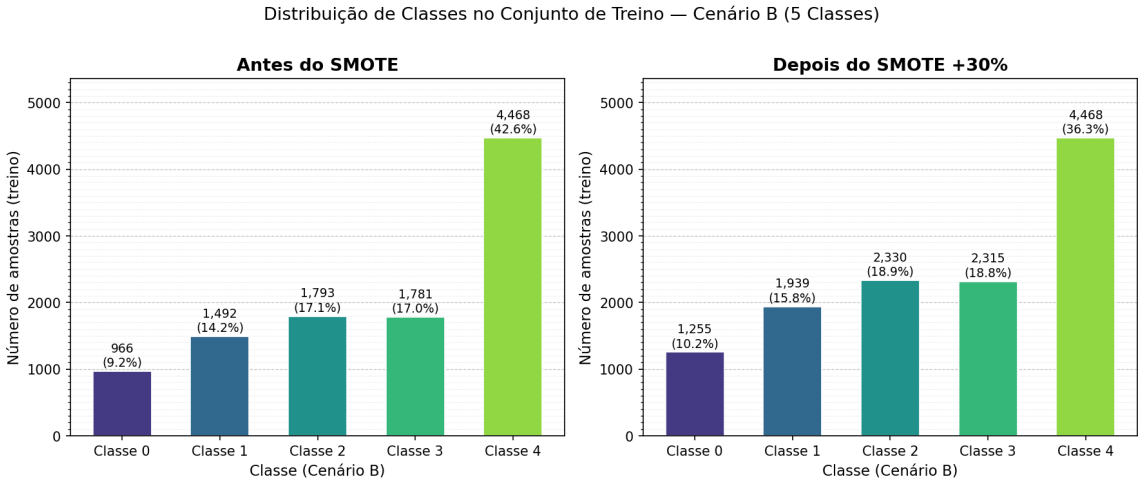


Figura 9: Distribuição de classes no conjunto de treino do Cenário B antes e depois do SMOTE +30%. A classe maioritária (classe 4) permanece inalterada; as restantes crescem proporcionalmente.

O Cenário A não necessita de SMOTE, uma vez que as classes estão perfeitamente balanceadas (50%/50%). Para os Cenários B e C, a análise dos resultados revela padrões distintos.

No **Cenário B**, o nível **+30%** produziu o melhor F1 no teste (0.7533), superando o baseline em cerca de 0.8 pontos percentuais. As melhorias são modestas mas consistentes, o que é esperado: com 5 classes e uma proporção de desequilíbrio de ≈1:4.6, o modelo já consegue aprender razoavelmente sem resampling.

No **Cenário C**, o SMOTE não produziu melhorias consistentes: o **baseline** obteve o melhor F1 no teste (0.4848) e nenhum nível de oversampling o superou de forma clara. Isto deve-se provavelmente ao facto de as classes mais pequenas (classe 0 com apenas 463 amostras de treino) terem muito poucos vizinhos reais, fazendo com que o SMOTE gere exemplos sintéticos ruidosos. O desequilíbrio residual é gerido nos modelos finais através de `class_weight='balanced'` no SVM e na NN, e de `sample_weight` no XGBoost.

4.3 Métricas de Avaliação

Tivemos de pensar cuidadosamente em quais as métricas mais adequadas para este problema. Desde logo, percebemos que usar apenas **accuracy** seria enganador com classes desbalanceadas, porque um modelo que preveja sempre a classe maioritária pode ter accuracy elevada mesmo sendo inútil. Por isso definimos o seguinte conjunto de métricas:

Cenário	Métrica Principal	Métricas Complementares
A (Binário)	F1-Score (macro)	Precision, Recall, ROC-AUC, MCC
B (5 Classes)	F1-Score (macro)	Weighted F1, Cohen's Kappa
C (10 Classes)	F1-Score (macro)	Top-2 Accuracy, Weighted F1, Cohen's Kappa

Escolhemos o **F1-Score macro** como métrica principal porque trata todas as classes de forma igualitária, independentemente da sua frequência, penalizando os modelos que ignoram classes minoritárias. O **Cohen's Kappa** complementa ao medir a concordância acima do que seria esperado por acaso. O **MCC** é particularmente robusto a

desequilíbrios. Para o Cenário C, acrescentámos ainda a **Top-2 Accuracy**, que considera uma previsão correta se a classe verdadeira estiver entre as duas mais prováveis, útil num problema de 10 classes onde o erro de um nível adjacente tem pouco significado prático.

4.4 Modelos Implementados

Para a escolha dos três modelos, o enunciado tornava o SVM e a Rede Neuronal obrigatórios. Para o terceiro, tivemos de justificar a nossa opção. Depois de pesquisar a literatura (em particular o trabalho de Huang et al. [10] que mostrou o XGBoost como melhor algoritmo num problema semelhante), optámos pelo XGBoost.

O XGBoost (*eXtreme Gradient Boosting*) [3] é um método de *ensemble* que combina sequencialmente várias árvores de decisão, onde cada árvore corrige os erros da anterior. Inclui regularização nativa para controlar o overfitting e tem demonstrado resultados consistentemente fortes em dados tabulares estruturados como os nossos [4].

A seguir descrevemos as nossas decisões de implementação para cada modelo.

4.4.1 Support Vector Machine (SVM)

O SVM [1] foi incluído por obrigação, mas também por convicção: trata-se de um modelo com fortes garantias teóricas (minimização do risco estrutural) que se adequa bem ao nosso problema. A razão principal é que os limites entre classes comportamentais como "stress moderado" e "stress alto" são intrinsecamente ruidosos e difusos, e a maximização da margem do SVM confere exactamente o tipo de robustez necessária nestes casos. Além disso, o kernel RBF permite capturar as interações não-lineares entre sono, ecrã e atividade física sem requerer que as criemos manualmente.

Implementação:

- Encapsulámos o modelo num `Pipeline([StandardScaler, SVC])` para prevenir data leakage durante a validação cruzada;
- Kernel RBF: mede a similaridade entre dois pontos com base na distância entre eles no espaço de features, permitindo capturar fronteiras não-lineares;
- `class_weight='balanced'` para compensar o desequilíbrio de classes;
- `probability=True` (necessário para calcular o ROC-AUC);
- Para o tuning, usámos **GridSearchCV** com 12 combinações de hiperparâmetros: regularização (C), largura do kernel (γ) e tipo de kernel (RBF vs. linear). Optámos pelo GridSearch em vez do RandomizedSearch porque o espaço é pequeno o suficiente para ser explorado exaustivamente em tempo razoável, garantindo que encontramos a melhor combinação e não apenas uma aproximação.

4.4.2 Rede Neuronal (MLPClassifier)

A MLP [2] foi o segundo modelo obrigatório. Incluímo-la porque oferece algo que os outros dois modelos não têm: a capacidade de aprender representações hierárquicas dos dados de forma completamente automática, o que é valioso quando as relações entre features são complexas e difíceis de especificar à mão.

Para a arquitetura, optámos por duas camadas ocultas (128 e 64 neurónios). Uma única camada pode não ter capacidade suficiente para aprender relações complexas entre

as features, enquanto arquiteturas muito profundas aumentam o risco de overfitting e o custo computacional sem benefício proporcional para um dataset desta dimensão. Duas camadas representam um equilíbrio razoável: a primeira aprende representações mais gerais dos dados e a segunda refina-as antes da classificação final.

Implementação:

- **Optimizer Adam** (`lr=0.001`): o otimizador é o algoritmo que controla como os pesos da rede são atualizados a cada iteração. O Adam ajusta automaticamente a taxa de aprendizagem para cada peso individualmente, o que acelera a convergência face a alternativas mais simples como o gradiente descendente clássico. O valor `lr=0.001` é o passo de atualização inicial, um valor padrão amplamente usado na literatura;

- **Regularização L2** (`alpha=1e-4`): penaliza pesos muito grandes na rede, forçando o modelo a soluções mais simples e menos propensas a memorizar o conjunto de treino em vez de generalizar. O valor `1e-4` é uma regularização leve, suficiente para estabilizar o treino sem comprometer a capacidade da rede;

- **Early Stopping** com *patience*=10: interrompe o treino automaticamente quando o desempenho na validação interna deixa de melhorar durante 10 épocas consecutivas. Evita que a rede continue a treinar depois de atingir o seu ponto ótimo, prevenindo overfitting sem necessidade de fixar manualmente o número de épocas;

- Para o tuning, escolhemos **RandomizedSearchCV** com 15 combinações: o espaço de hiperparâmetros de uma rede neuronal (neurónios, taxa de aprendizagem, regularização) é demasiado vasto para explorar exaustivamente, e a pesquisa aleatória é empiricamente eficiente nestes casos.

4.4.3 XGBoost: 3.º Modelo

O XGBoost [3] foi o nosso modelo de escolha livre, e fizemo-lo por quatro razões que se complementam:

1. **Adequação a dados tabulares**: os nossos dados têm exactamente o formato em que o XGBoost costuma destacar-se (tabular, estruturado, com features numéricas e categóricas), como demonstrado empiricamente por Shwartz-Ziv & Armon [4];

2. **Robustez sem normalização**: ao contrário do SVM, o XGBoost não precisa que as features estejam na mesma escala, o que simplifica o pipeline;

3. **Interpretabilidade via SHAP**: dado que o nosso problema tem potencial aplicação em contextos de saúde, quisemos poder explicar cada previsão individualmente e o XGBoost facilita isso de forma nativa através dos SHAP values. Os SHAP values (*SHapley Additive exPlanations*) quantificam a contribuição de cada feature para uma previsão concreta: um valor positivo indica que aquela feature empurrou a previsão para uma classe de stress mais alta, e um valor negativo para uma mais baixa. Isto permite explicar individualmente cada caso em vez de apenas reportar métricas globais;

4. **Evidência na literatura**: Huang et al. [10] mostraram que o XGBoost supera SVM, KNN e ANN num problema quase idêntico ao nosso.

Implementação:

- `objective='multi:softprob'` para multiclasse;
- `sample_weight='balanced'` para compensar o desequilíbrio;
- `early_stopping_rounds` para parar quando o modelo deixar de melhorar;
- **RandomizedSearchCV** com 20 combinações aleatórias, pela mesma razão da MLP: o espaço de hiperparâmetros é demasiado grande para exploração exaustiva.

4.4.4 Ensemble de Redes Neuronais (PyTorch)

Como extensão ao MLPClassifier do scikit-learn, implementámos um **ensemble de redes neuronais em PyTorch** [5]. A motivação é dupla: por um lado, o PyTorch permite adicionar **Dropout** real entre camadas, que é uma forma de regularização mais eficaz do que a L2 isolada (o Dropout desativa aleatoriamente neurónios durante o treino, obrigando a rede a aprender representações redundantes e mais robustas); por outro, um ensemble de múltiplas redes reduz a variância das previsões.

O ensemble é composto por **5 redes com arquiteturas diversas** para maximizar a diversidade das previsões, cobrindo o mesmo espaço explorado pelo RandomizedSearchCV da MLP individual:

Tabela 6: Configurações das 5 redes do ensemble PyTorch.

Rede	Camadas ocultas	LR	Dropout
1	(128, 64)	0.001	0.1
2	(256, 128, 64)	0.001	0.1
3	(128, 64)	0.0005	0.2
4	(64, 32)	0.001	0.1
5	(256, 128)	0.001	0.1

A previsão final é a média das probabilidades *softmax* das 5 redes. Todas as redes usam o otimizador Adam com `weight_decay=1e-4` (regularização L2) e early stopping com *patience*=20. As configurações são as mesmas para os três cenários, sem tuning por cenário.

4.5 Validação Cruzada

Para avaliar os modelos de forma robusta antes de avançar para o tuning de hiperparâmetros, aplicámos **Stratified K-Fold Cross-Validation** com $k = 5$. A escolha do $k = 5$ foi um compromisso entre custo computacional (com 3 modelos $\times$ 3 cenários, um k maior tornava o processo muito lento) e robustez estatística.

A palavra-chave *stratified* foi fundamental na nossa implementação: garante que cada fold mantém a mesma proporção de classes que o dataset completo, evitando o risco de por azar concentrar todas as instâncias de uma classe rara num único fold. O resultado que reportamos para cada modelo é a média e o desvio padrão dos 5 valores de F1 obtidos, o que nos dá não só uma estimativa de desempenho mas também uma medida da sua estabilidade.

Os resultados da fase *Baseline* (com hiperparâmetros default, antes de qualquer tuning) são os seguintes:

Tabela 7: Cross-Validation Baseline: F1 Macro ($\mu \pm \sigma$, 5 folds)

Modelo	**Cenário A**	**Cenário B**	**Cenário C**
SVM	0.9458 ± 0.0044	0.8057 ± 0.0097	0.5647 ± 0.0053
NN/MLP	0.9453 ± 0.0063	0.8051 ± 0.0093	0.5701 ± 0.0150
XGBoost	0.9451 ± 0.0057	0.8080 ± 0.0096	0.5739 ± 0.0111

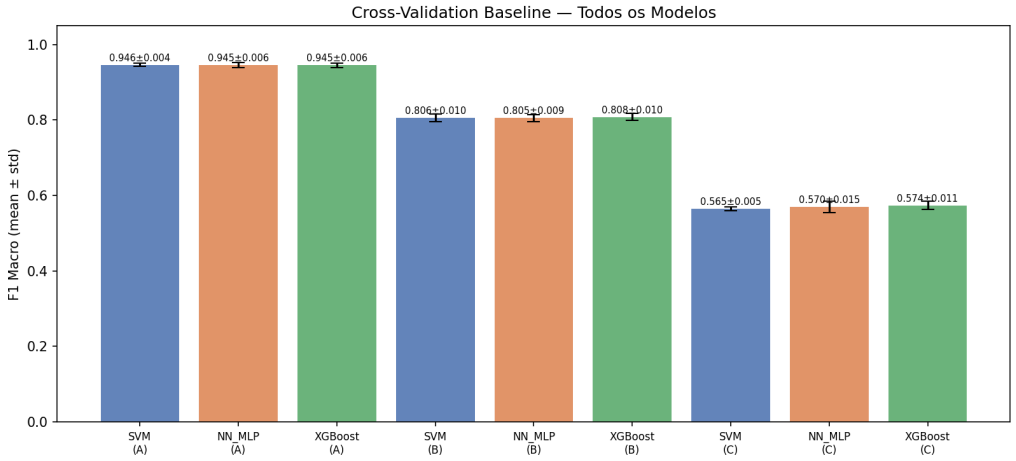


Figura 10: Resultados do CV Baseline para todos os modelos e cenários.

O desvio padrão baixo em todos os casos (≤ 0.015) foi um sinal positivo: os modelos comportam-se de forma estável e consistente entre folds, o que nos deu confiança para avançar para o tuning de hiperparâmetros.

5 Resultados e Discussão

5.1 Resultados no Conjunto de Teste (Hold-out)

Depois de treinar e afinar todos os modelos, chegou o momento de os avaliar sobre o conjunto de teste: as 2 250 instâncias que tínhamos reservado desde o início e que nenhum modelo alguma vez tinha visto. Esta é a avaliação que consideramos definitiva, porque reflete o desempenho real em dados completamente novos.

Tabela 8: Resultados finais no conjunto de teste hold-out: métricas por modelo e cenário.

Modelo	Cenário	F1 macro	Accuracy	Cohen's κ	MCC	ROC-AUC
NN/MLP	A	**0.9502**	**0.9502**	0.9004	0.9006	0.9906
SVM	A	0.9476	0.9476	0.8951	0.8951	**0.9916**
XGBoost	A	0.9462	0.9462	0.8924	0.8925	0.9905
SVM	B	**0.8229**	**0.8418**	**0.7817**	0.7823	**0.9683**
XGBoost	B	0.8201	0.8391	0.7778	0.7782	0.9656
NN/MLP	B	0.8124	0.8351	0.7709	0.7710	0.9658
XGBoost	C	**0.5819**	**0.6382**	**0.5721**	**0.5731**	0.9336
SVM	C	0.5766	0.6284	0.5615	0.5624	**0.9389**
NN/MLP	C	0.5700	0.6351	0.5655	0.5656	0.9361

Tabela Comparativa Final — Fase 3

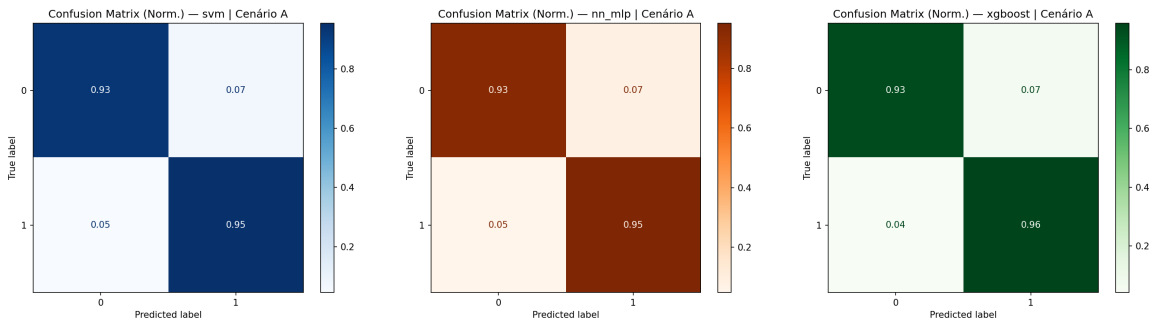
model	scenario	F1-macro (CV)	F1-macro (Test)	Accuracy	Cohen's κ	MCC	ROC-AUC
NN	A	0.9445 ± 0.0051	0.9502	0.9502	0.9004	0.9006	0.9906
SVM	A	0.9477 ± 0.0052	0.9476	0.9476	0.8951	0.8951	0.9916
XGB	A	0.9466 ± 0.0063	0.9462	0.9462	0.8924	0.8925	0.9905
NN	B	0.8056 ± 0.0106	0.8124	0.8351	0.7709	0.771	0.9658
SVM	B	0.8144 ± 0.0118	0.8229	0.8418	0.7817	0.7823	0.9683
XGB	B	0.8085 ± 0.0078	0.8201	0.8391	0.7778	0.7782	0.9656
NN	C	0.5738 ± 0.0120	0.57	0.6351	0.5655	0.5656	0.9361
SVM	C	0.5907 ± 0.0087	0.5766	0.6284	0.5615	0.5624	0.9389
XGB	C	0.5686 ± 0.0138	0.5819	0.6382	0.5721	0.5731	0.9336

Figura 11: Visualização gráfica da tabela comparativa final (hold-out test set).

5.2 Matrizes de Confusão

Para além dos números agregados, analisámos as matrizes de confusão de cada modelo em cada cenário, com o objetivo de perceber *onde* especificamente os modelos erram e não apenas *quanto*.

5.2.1 Cenário A: Classificação Binária

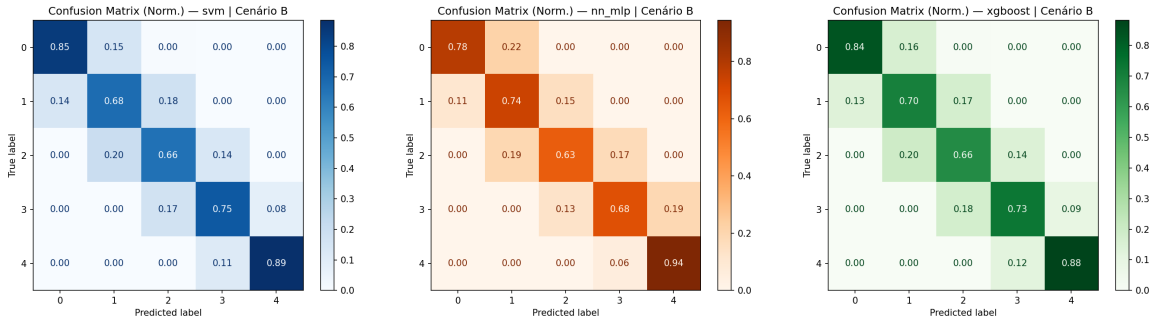


(a) SVM (b) NN/MLP (c) XGBoost

Figura 12: Confusion matrices normalizadas: Cenário A (Binário).

As matrizes do Cenário A revelam um desempenho muito uniforme entre os três modelos. A diagonal principal está claramente dominante em todos os casos, com taxas de acerto entre 94% e 96% para ambas as classes. O padrão de erros é simétrico: os falsos positivos e falsos negativos têm magnitudes comparáveis, o que confirma que o uso de `class_weight='balanced'` funcionou como esperado e que os modelos não estão enviesados para nenhuma das classes.

5.2.2 Cenário B: 5 Classes

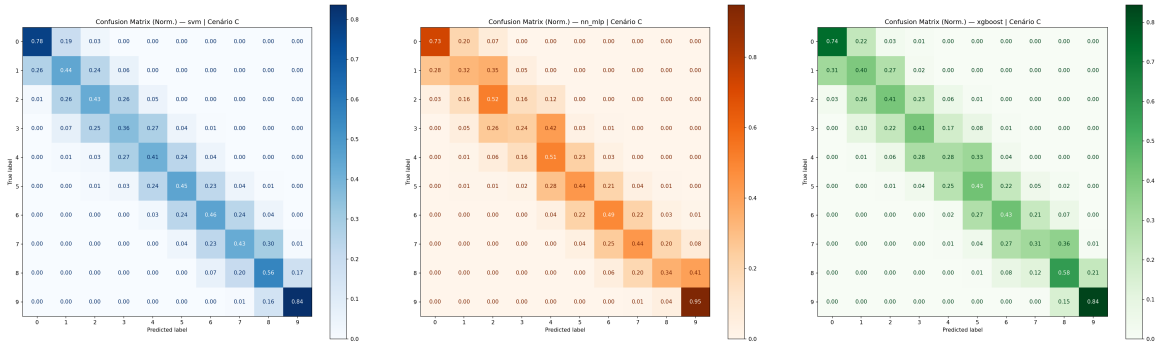


(a) SVM (b) NN/MLP (c) XGBoost

Figura 13: Confusion matrices normalizadas: Cenário B (5 Classes).

No Cenário B, as matrizes expõem um padrão claro e esperado: as **classes extremas (0 e 4)** são classificadas com mais precisão (a diagonal nestas linhas é mais intensa), enquanto as **classes intermédias (1, 2 e 3)** mostram confusão maioritariamente *lateral* (entre classes vizinhas), raramente saltando mais de uma posição. Este comportamento é semanticamente coerente: distinguir stress "sem" de stress "extremo" é muito mais fácil do que distinguir "baixo" de "moderado", onde as diferenças comportamentais são subtis. A SVM, que obteve o melhor resultado neste cenário, apresenta a diagonal mais limpa das três.

5.2.3 Cenário C: 10 Classes



(a) SVM (b) NN/MLP (c) XGBoost

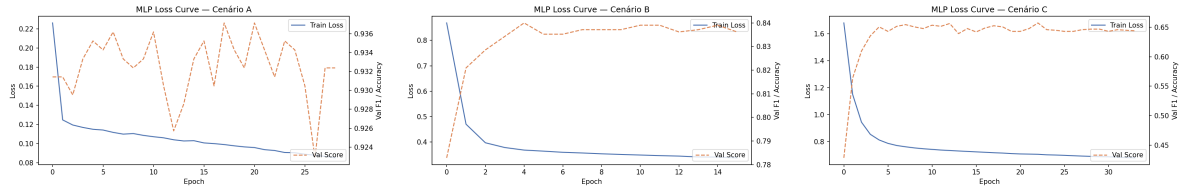
Figura 14: Confusion matrices normalizadas: Cenário C (10 Classes).

No Cenário C, as matrizes ficam visivelmente mais densas, mas o padrão de erros tem uma estrutura importante: é **diagonal-bandlike**, ou seja, os erros concentram-se quase

exclusivamente nas classes adjacentes e não em saltos arbitrários entre classes distantes. Isto revela que os modelos capturam a natureza ordinal do stress: sabem que nível 5 está "próximo" de nível 6, mesmo sem qualquer indicação explícita desta ordem durante o treino. A diagonal principal, ainda que menos intensa do que nos cenários anteriores, permanece a região mais escura de cada linha, confirmando que os modelos aprendem de facto.

5.3 Curvas de Loss da Rede Neuronal

Para a rede neuronal, monitorizar a curva de loss ao longo do treino foi importante para verificar se o modelo estava a convergir de forma estável ou se apresentava sinais de overfitting. A curva de loss regista, época a época, o erro cometido pelo modelo durante o treino: idealmente, esse erro desce progressivamente à medida que a rede aprende. Quando a loss de treino continua a descer mas a loss de validação começa a subir, isso é sinal de overfitting — o modelo está a memorizar o treino em vez de generalizar. Ficámos globalmente satisfeitos com o que observámos, embora com nuances por cenário.



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 15: Curvas de loss da MLP nos três cenários. A loss de treino converge de forma estável em todos os cenários; o val score no Cenário A apresenta oscilações mas sem divergência.

Nos três cenários, a loss de treino desce de forma estável e não se observa divergência entre treino e validação, o que exclui overfitting grosseiro. No entanto, o comportamento não é uniforme: no **Cenário A**, o val score (linha laranja) apresenta oscilações consideráveis ao longo das épocas, refletindo a instabilidade natural da estimativa de F1 num conjunto de validação interno pequeno com apenas duas classes — ainda assim, a loss de treino converge sem problemas e o desempenho final é elevado. Nos **Cenários B e C**, o val score estabiliza de forma mais suave, o que é coerente com a maior dimensão do problema de classificação. No Cenário C, a loss de validação estabiliza num valor mais elevado do que nos restantes, refletindo a maior dificuldade intrínseca do problema com 10 classes.

5.4 Análise Crítica por Cenário

5.4.1 Cenário A: Classificação Binária

Ficámos genuinamente satisfeitos com os resultados do Cenário A: os três modelos atingem F1 macro entre 0.946 e 0.950, o que representa um **desempenho excelente** para um problema com dados sintéticos. A **NN/MLP** surpreendeu-nos ao superar marginalmente os outros dois (F1=0.9502), seguida do SVM (0.9476) e do XGBoost (0.9462).

Ao inspecionar as matrizes de confusão, verificámos que os erros são simétricos entre as duas classes (≈5% FP e ≈4% FN), ou seja, o modelo não está sistematicamente a

errar mais numa direção do que na outra. Isso confirmou que a nossa decisão de usar `class_weight='balanced'` foi acertada. Comparando com a literatura (Tabela 1), os nossos 95% ficam ligeiramente abaixo dos 98% de Lei et al. [11], o que é expectável dado que o seu dataset apresenta maior separabilidade de classes.

5.4.2 Cenário B: 5 Classes

No Cenário B, o desempenho desceu para F1 $\approx$ 0.81–0.82, o que consideramos razoável para um problema de 5 classes. Ao analisar as matrizes de confusão, encontrámos um padrão interessante que nos ajudou a perceber melhor o problema:

- As **classes extremas (0 e 4)** são classificadas com muito mais precisão ($\approx$87–95%), porque têm perfis de features muito distintos dos restantes;
- As **classes intermédias (1, 2 e 3)** são confundidas entre si (F1$\approx$0.72–0.73), o que faz sentido: uma pessoa com stress "baixo" e uma com stress "moderado" têm comportamentos muito semelhantes e a fronteira entre elas é genuinamente ambígua.

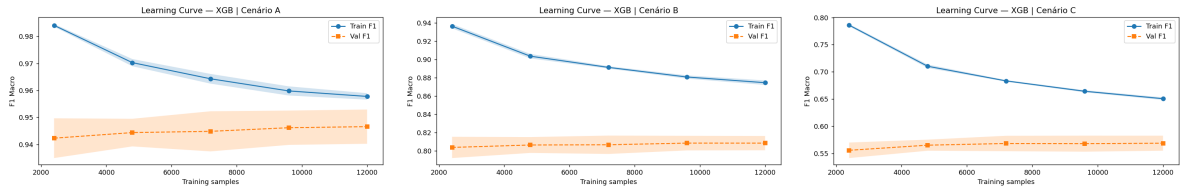
O facto de o **SVM** ter obtido o melhor resultado no Cenário B (F1=0.8229) foi uma surpresa. Verificámos que o GridSearch selecionou o **kernel linear** em vez do RBF default, o que sugere que, após normalização, o espaço de features é aproximadamente linearmente separável para as 5 classes, e que a margem máxima do SVM linear se revelou mais robusta do que as fronteiras não-lineares do XGBoost neste contexto.

5.4.3 Cenário C: Trade-offs de Granularidade (10 Classes)

O Cenário C foi o mais desafiante e, honestamente, o mais revelador. O F1 macro desceu para $\approx$0.57–0.58, o que à primeira vista pode parecer fraco, mas que analisámos criticamente e entendemos como resultado de um **trade-off inerente entre granularidade e precisão**. A queda explica-se por dois fatores:

1. **Sobreposição semântica**: distinguir stress de "nível 5" de "nível 6" é uma tarefa que nem um humano conseguiria fazer com certeza a partir de variáveis comportamentais. Os modelos têm de "memorizar" diferenças de fronteira ínfimas, o que cria instabilidade;
2. **Fronteiras de igual largura**: o equal-width binning cria limites geometricamente uniformes mas semanticamente arbitrários nas zonas intermédias, forçando o modelo a distinguir casos onde a diferença real de comportamento é quase nula.

Para perceber melhor o problema, analisámos as **Learning Curves** do XGBoost (Figura 16), que confirmaram que o limite de desempenho do Cenário C ($\sim$0.60) é intrínseco ao problema e não seria superado com mais dados. A ausência de divergência entre treino e validação excluiu overfitting como causa. Um dado reconfortante: a **Top-2 Accuracy** supera os 85%, o que significa que o modelo raramente erra por mais de um nível, um erro aceitável numa escala de 10.



(a) Cenário A (b) Cenário B (c) Cenário C

Figura 16: Learning Curves do XGBoost. O patamar assintótico decresce do Cenário A (∼0.94) para o C (∼0.60), confirmando que o trade-off de granularidade é intrínseco ao problema e não resolúvel com mais dados.

5.5 Comparação entre Modelos

Olhando para os resultados de conjunto, cada modelo mostrou um perfil diferente que nos levou a conclusões concretas:

- **NN/MLP**: foi o melhor no Cenário A (F1=0.9502), o que nos surpreendeu positivamente dado que usámos uma arquitetura relativamente simples (2 camadas ocultas, regularização L2). Acreditamos que uma arquitetura mais profunda com Dropout real melhoraria os resultados nos Cenários B e C, mas ficámos satisfeitos com o que a MLP do scikit-learn conseguiu alcançar.
- **SVM**: foi o melhor no Cenário B (F1=0.8229) e manteve-se sempre competitivo. A sua principal desvantagem que observámos foi o custo computacional sendo que o treino foi significativamente mais lento do que os outros dois modelos, especialmente com kernel RBF ($O(n^2)$).
- **XGBoost**: foi o melhor no Cenário C (F1=0.5819) e segundo no A. Foi também o mais rápido e estável a treinar, graças ao early stopping e à invariância à escala. No geral, confirmou as expectativas que tínhamos criado com base na literatura.

5.6 Efeito do Tuning de Hiperparâmetros

Uma observação que consideramos importante partilhar é que a melhoria obtida com o tuning de hiperparâmetros foi modesta, tipicamente entre +0.5% e +1.5% em F1. Isso diz-nos que os hiperparâmetros default dos algoritmos já são razoáveis para este tipo de dataset, e que o esforço de otimização não foi o fator determinante na qualidade dos resultados. A contribuição mais valiosa do GridSearch/RandomizedSearch foi a seleção do **kernel linear no SVM** em detrimento do RBF default, que se revelou mais adequado para os Cenários B e C.

5.7 Ensemble de Redes Neuronais (PyTorch)

A Tabela 9 compara o ensemble PyTorch com a MLP individual do scikit-learn no conjunto de teste.

Tabela 9: Comparação do ensemble PyTorch com a MLP sklearn no conjunto de teste (F1 macro).

Modelo	**Cenário A**	**Cenário B**	**Cenário C**
NN/MLP sklearn (tuned)	**0.9502**	0.8124	0.5700
Ensemble PyTorch	0.9489	0.7572	0.4932

No **Cenário A**, o ensemble atinge 0.9489, praticamente idêntico à MLP individual (diferença de 0.001), confirmando que os dois modelos capturam o problema binário com a mesma eficácia. Nos **Cenários B e C**, o ensemble fica abaixo da MLP sklearn. O tuning específico por cenário não foi realizado por duas razões: o ensemble é uma das últimas matérias lecionadas na disciplina, pelo que a sua implementação ocorreu nos últimos dias do projeto, não deixando tempo útil suficiente para realizar o tuning; além disso, afinar um ensemble de redes neuronais exige um poder computacional e um número de treinos consideravelmente superior ao de um modelo individual. As configurações fixas usadas representam uma escolha razoável mas não otimizada, sendo o tuning específico por cenário identificado como trabalho futuro.

5.8 Interpretabilidade Local (SHAP)

Sendo um problema com potencial relevância na área da saúde, quisemos ir além das métricas globais e perceber *porquê* os modelos fazem as previsões que fazem. Para isso, aplicámos **SHAP (SHapley Additive exPlanations)** ao XGBoost, que nos permite ver o impacto de cada feature em cada instância individual.

O resultado (Figura 17) confirmou a nossa intuição: o `mental_fatigue_score` é o preditor de longe mais impactante: valores elevados empurram consistentemente a previsão para uma classe de stress superior. Mas o que nos agradou particularmente foi ver que os rácios `ratio_usage_sleep` e `ratio_activity_usage`, as features que criámos nós próprios, também emergem como relevantes. Isso validou retroativamente a nossa decisão de os criar.

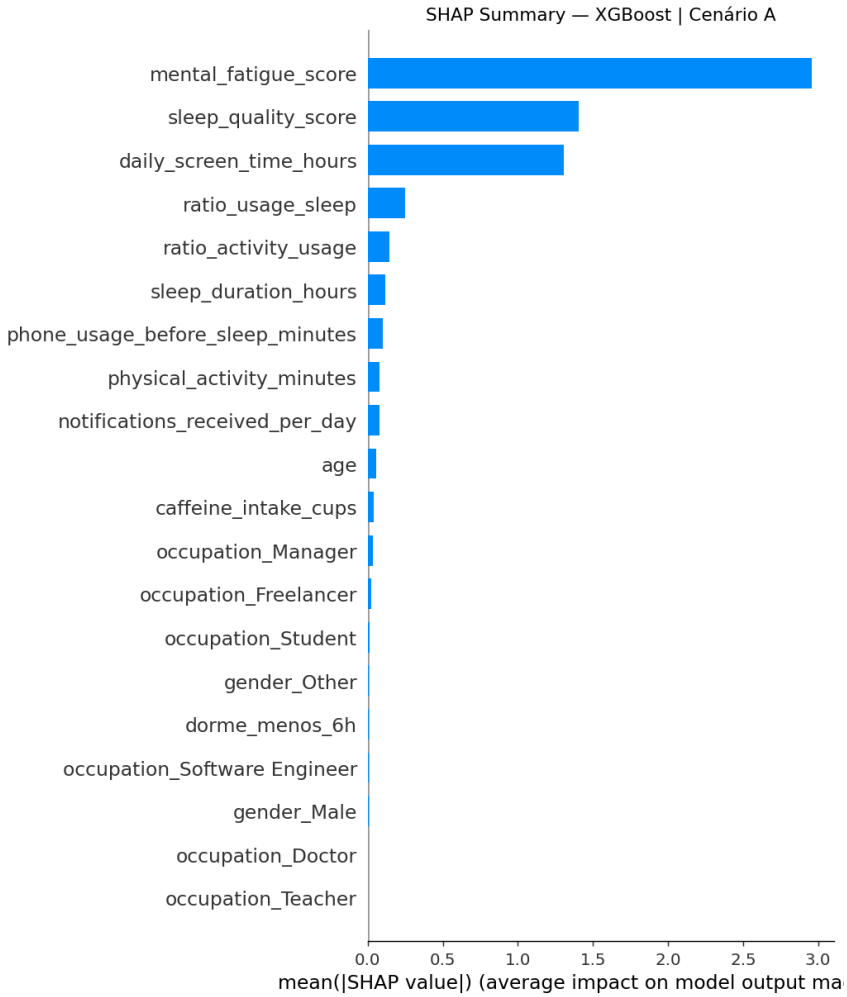


Figura 17: SHAP Summary Plot: XGBoost, Cenário A. Cada ponto representa uma instância; a posição horizontal indica a direção e magnitude do impacto no output do modelo.

5.9 Testes de Significância Estatística

Uma questão que colocámos a nós próprios foi: as diferenças que observamos entre modelos são reais, ou poderiam ser produto de variância aleatória? Para responder rigorosamente, aplicámos dois testes estatísticos complementares.

Teste de McNemar (comparação por pares sobre o conjunto de validação):

Tabela 10: Resultados do teste de McNemar: valor-p (exact). $\star = p < 0.05$.

Par	Cenário A	Cenário B	Cenário C
SVM vs NN	0.3581	0.1014	**0.0440** $\star$
SVM vs XGB	0.2962	0.6173	0.5088
NN vs XGB	0.0912	0.3723	0.2433

O único resultado estatisticamente significativo que encontrámos foi a diferença entre **SVM e NN no Cenário C** ($p = 0.044$): estes dois modelos erram em instâncias diferentes de forma assimétrica, o que não é explicável por acaso. Nos restantes cenários, os

três modelos não diferem significativamente, o que é consistente com a proximidade dos seus valores de F1 e nos disse que, para os Cenários A e B, qualquer dos três modelos seria igualmente válido para produção.

Teste de Wilcoxon (sobre os scores dos 5 folds de CV):

Tabela 11: Resultados do teste de Wilcoxon com $k = 5$ folds. Nenhum par atinge $p < 0.05$.

Par	**Cenário A**	**Cenário B**	**Cenário C**
SVM vs NN	0.0625	0.0625	0.0625
SVM vs XGB	0.1250	0.1250	0.0625
NN vs XGB	0.3125	0.3125	0.6250

Com apenas 5 folds, percebemos que o teste de Wilcoxon só pode produzir valores de p discretos (0.0625, 0.125, ...), o que torna impossível atingir o limiar de 0.05 – uma limitação que reconhecemos desde o início. Para termos maior poder estatístico neste teste, teríamos de usar $k \geq 10$ folds, o que foi inviável pelo custo computacional do SVM com 3 cenários. Ainda assim, decidimos incluir os resultados por transparência.

5.10 Comparação com Trabalhos Existentes

Com os resultados obtidos, podemos agora fazer uma comparação mais informada com os trabalhos que consultámos no Background.

Relativamente ao trabalho de Tolstoyj [9], a diferença de abordagem é clara: onde ele tentou prever o valor numérico exato do stress (**regressão**) e concluiu que as relações eram fracas, nós convertemos o stress contínuo em categorias e tratámos o problema como **classificação**. O nosso F1 macro de ≈0.95 no Cenário A mostra que a fraqueza reportada por Tolstoyj não era uma característica do dataset, mas sim uma limitação da abordagem de regressão. Um modelo que só precisa de decidir "esta pessoa está no grupo de stress alto ou baixo" tem uma tarefa muito mais tratável do que um que tem de acertar num valor preciso como 7.38.

No que toca à literatura académica, os nossos resultados no Cenário A (F1 = 0.9502) ficam próximos dos 0.98 reportados por Lei et al. [11] com Random Forest. A diferença é pequena e explicável: o dataset de Lei et al. contém apenas variáveis de sono, o que torna as classes mais facilmente separáveis. O nosso dataset inclui também tempo de ecrã e atividade física, variáveis que, apesar de informativas, tornam o espaço de decisão mais complexo. Comparativamente ao trabalho de Huang et al. [10] (XGBoost, 90.66%), os nossos modelos ficam acima, o que é encorajador dado que utilizámos dados e variáveis estruturalmente semelhantes. O resultado de Acosta et al. [12] (>98%) é claramente superior, mas é uma comparação injusta: sensores físicos contínuos (frequência cardíaca, temperatura, saturação de oxigénio) são muito mais ricos em informação do que registos de comportamento como os nossos.

5.11 Limitações e Desafios

Sendo honestos sobre o trabalho que fizemos, identificamos três limitações principais que afetam a generalização das conclusões.

Em primeiro lugar, o facto de os dados serem **sintéticos** facilita artificialmente a separabilidade das classes. Os 95% de F1 que obtivemos no Cenário A não se traduziria

provavelmente com a mesma magnitude em dados reais de wearables, onde há ruído, valores em falta e subjetividade na medição do stress.

Em segundo lugar, a **subjetividade inerente à variável-alvo** é um problema que não conseguimos resolver: o stress é uma perceção pessoal, e dois indivíduos com exatamente o mesmo comportamento podem classificar o seu stress de forma completamente diferente. Isso cria um teto de Bayes Error estrutural que nenhum algoritmo consegue superar.

Em terceiro lugar, o **Cenário C com 10 classes** é conceptualmente questionável para um problema inerentemente contínuo: uma abordagem de **regressão ordinal** seria provavelmente mais adequada, e é algo que exploraríamos num trabalho futuro.

6 Conclusões

Olhando para o trabalho no seu conjunto, sentimos que atingimos os objetivos que tínhamos proposto. Aplicámos três modelos de ML distintos a um problema de classificação de stress em três cenários de complexidade crescente, e fizemo-lo de forma sistemática e com rigor metodológico.

Os principais resultados que obtivemos foram:

- F1 macro de ≈0.95 no Cenário A, onde a **NN/MLP** superou ligeiramente os outros dois (0.9502), o que foi uma surpresa positiva, e coloca o nosso trabalho a par da literatura para datasets sintéticos comportamentais;
- F1 macro de ≈0.82 no Cenário B, onde o **SVM** se destacou (0.8229), com a confusão esperada nas classes intermédias que identificámos e explicámos;
- F1 macro de ≈0.58 no Cenário C, onde o **XGBoost** liderou (0.5819), mas reconhecemos que este cenário representa um problema difícil por natureza e não por falha dos modelos.

Os testes de McNemar que realizámos revelaram que apenas a diferença SVM vs. NN no Cenário C é estatisticamente significativa ($p = 0.044$); nos outros cenários, os três modelos são equivalentes entre si. Comparando com a literatura, os nossos resultados são consistentes com os trabalhos sobre dados sintéticos de stress/sono [11], embora naturalmente abaixo dos obtidos com sinais fisiológicos reais [12].

Se voltássemos a fazer este projeto, exploraríamos: (i) validação com dados reais de wearables para testar a robustez das conclusões; (ii) arquiteturas MLP mais profundas com Dropout (via PyTorch) para melhorar o Cenário B; (iii) reformulação do Cenário C como regressão ordinal, que seria semanticamente mais adequado para uma escala contínua.

Utilização de Inteligência Artificial

No desenvolvimento deste projeto foi utilizado o modelo **Claude Sonnet 4.7** da **Anthropic** como ferramenta de apoio para auxiliar e otimizar o desenvolvimento do projeto. Todas as decisões metodológicas, interpretações e conclusões foram revistas e validadas pelos autores.

Referências

[1] Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[2] Rumelhart, D., Hinton, G. & Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[3] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[4] Shwartz-Ziv, R. & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[5] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

[6] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[7] JayJoshi37 (2024). Sleep, Screen Time and Stress Analysis. Kaggle. `https://www.kaggle.com/datasets/jayjoshi37/sleep-screen-time-and-stress-analysis`

[8] Badrnezhad, H. (2024). Screen Time, Sleep and Stress Analysis Dataset – ETL Pipeline. Kaggle Notebook. `https://www.kaggle.com/code/hosseinbadrnezhad/screen-time-sleep-and-stress-analysis-dataset-etl`

[9] Tolstoy, J. (2024). Screen Time, Sleep & Stress Analysis – EDA and ML Notebooks. GitHub. `https://github.com/Tolstoyj/screen-time-sleep-stress-analysis`

[10] Huang, Y. et al. (2024). Applying Machine Learning Algorithms for the Classification of Sleep Disorders. *International Journal of Engineering and Science Research (IJESR)*. `https://www.ijesr.org/index.php/ijesr/article/view/321`

[11] Lei, X. et al. (2024). Prediction of Stress Levels in Sleep Patterns Based on Random Forest. *Applied and Computational Engineering*, 10280. `https://ace.ewapub.com/article/view/10280`

[12] Acosta, R. et al. (2025). Predicting Stress During Sleep from Biosensor Data: An Optimized Machine Learning Framework. *Journal of Human, Earth, and Future*. `https://hefjournal.org/index.php/HEF/article/view/575`